

PROYECTO INTERMODULAR 2

SPRINT 3 - Implementación Completa del Módulo de Auditoría

Miriam Mendieta Racedo.-
2DAMS
31/10/2026

SPRINT 3 - Implementación Completa del Módulo de Auditoría

- Proyecto: ZeertyDigital.
- Autor@: Miriam Mendieta Racedo.-
- Fecha: 31 de octubre de 2026
- IES Serra Perenxisa - Torrent

INDICE

INTRODUCCIÓN ---------- **1**
Diagrama de Casos de Uso – Auditoría ---------- **2**
Diagrama de Actividades – Flujo de Auditoría ---------- **3**
Diagrama de Clases UML – Módulo de Auditoría ---------- **4**
DIAGRAMA ER COMPLETO ---------- **5**
ARQUITECTURA DEL MÓDULO ---------- **7**
BIBLIOGRAFÍA ---------- **17**

INTRODUCCIÓN

En los sistemas de gestión de activos digitales, la **auditoría** es un componente esencial para garantizar la **trazabilidad**, la **seguridad**, la **integridad de los datos** y el **control interno**. Este módulo permite registrar de forma **cronológica, inmutable y detallada** todas las acciones realizadas por los usuarios sobre los activos: creación, edición, eliminación, visualización, descargas, subidas de documentos y cambios de estado.

La auditoría no solo sirve para detectar errores o accesos indebidos, sino que también es una herramienta clave para:

- **Cumplimiento normativo**.
- **Prevención de fraude o manipulación de información**.
- **Análisis de actividad y comportamiento de usuarios**.
- **Responsabilidad y transparencia en la gestión de activos**.

En este contexto, el rol **Auditor** adquiere una función trascendental: supervisar, revisar y validar que todas las operaciones realizadas en el sistema cumplen con las políticas internas y los estándares de seguridad establecidos. El Auditor no modifica datos; **observa, analiza y garantiza la integridad del sistema**.

El módulo de Auditoría permite registrar, consultar y supervisar todas las acciones realizadas sobre los activos digitales del sistema. Solo los roles ADMINISTRACIÓN y AUDITORÍA pueden acceder al registro de movimientos.

Este módulo es especialmente importante en aplicaciones ya que los activos digitales representan información sensible, documentos oficiales, recursos corporativos o elementos sujetos a control administrativo.

Diagrama de Casos de Uso – Auditoría

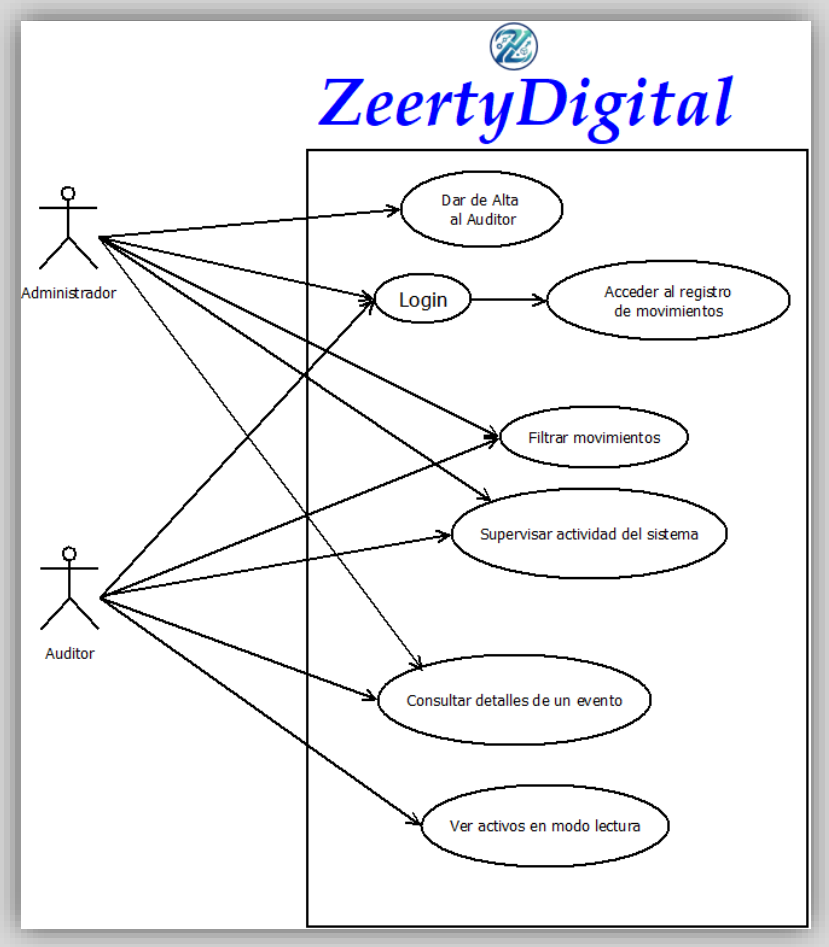


Diagrama de Actividades – Flujo de Auditoría

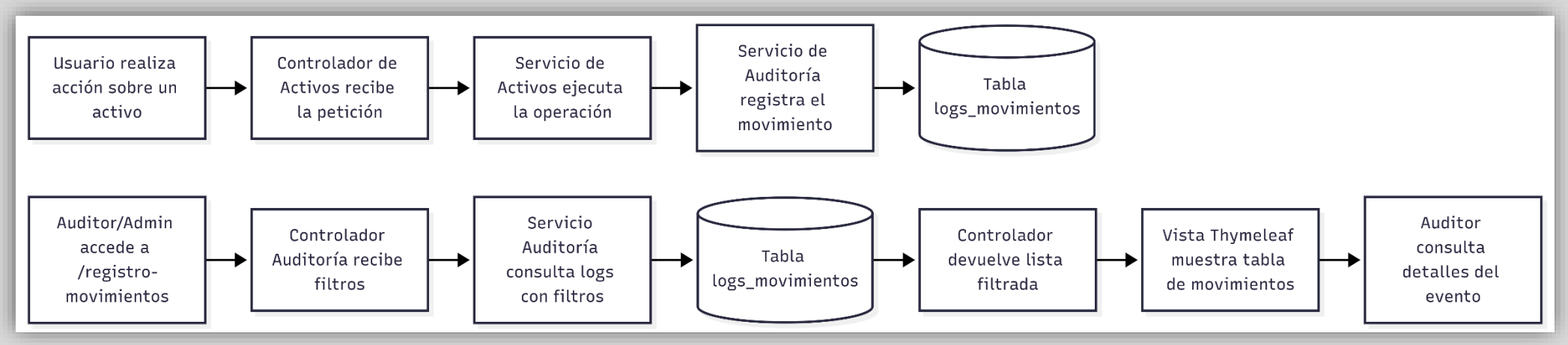


Diagrama de Clases UML – Módulo de Auditoría

Representa las entidades y relaciones principales.

Relación:

- Un **Usuario** genera muchos movimientos.
- Un **ActivoDigital** puede tener muchos movimientos asociados.

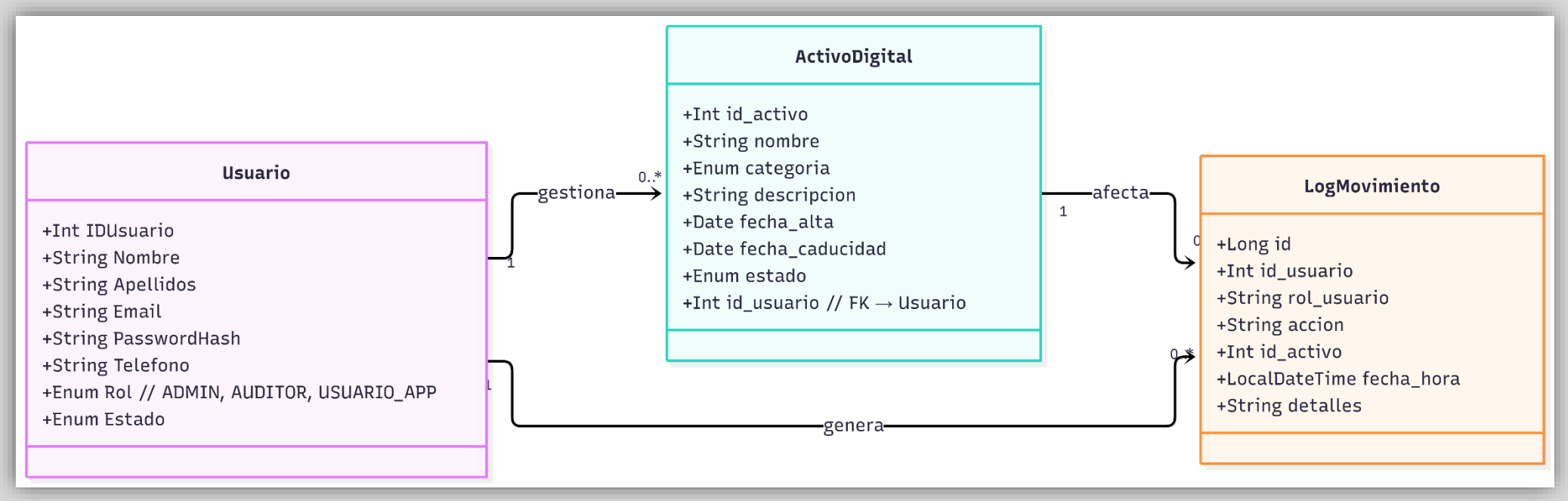
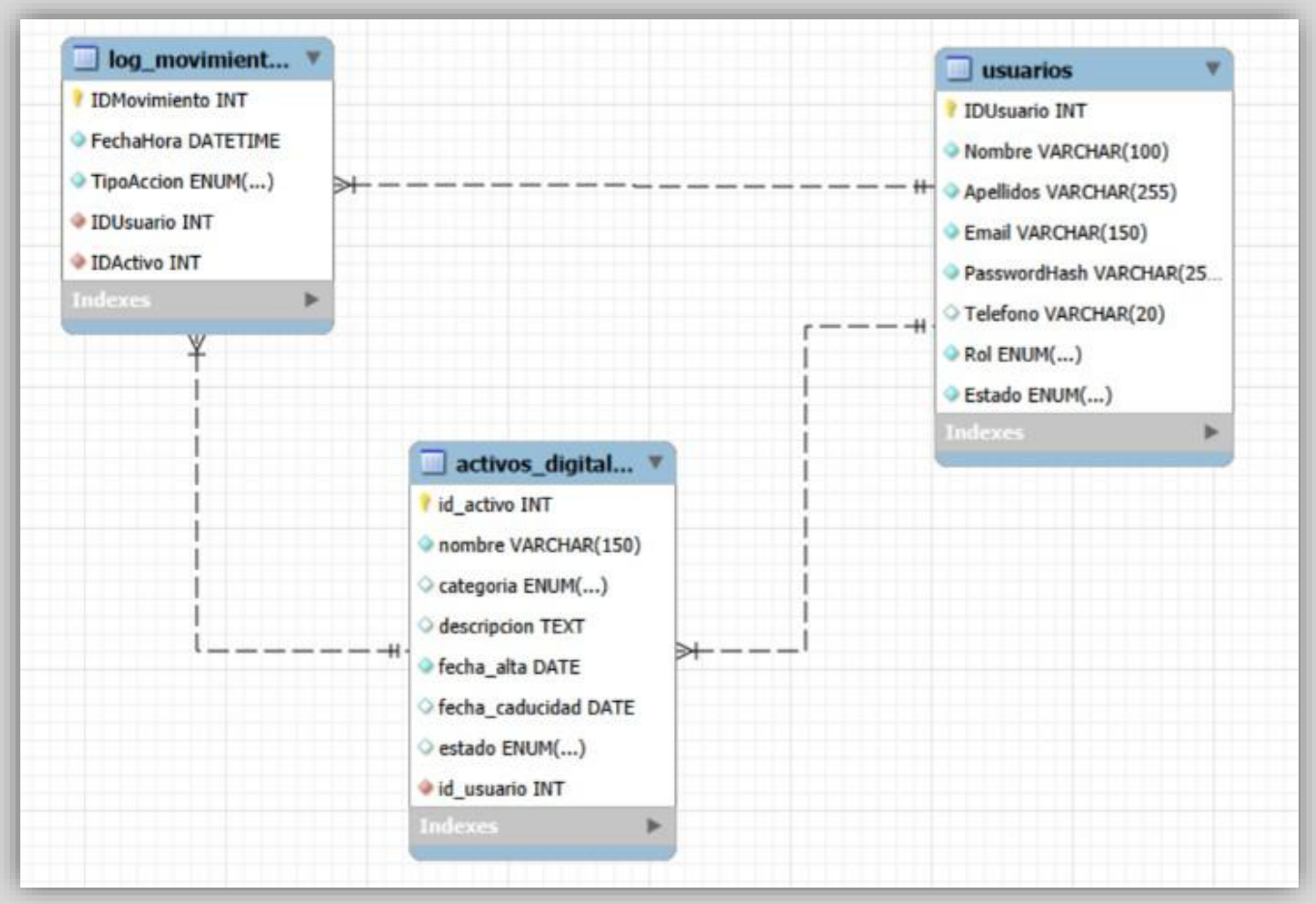


DIAGRAMA ER COMPLETO



Relaciones

- Un **usuario** puede generar muchos movimientos.
- Un **activo** puede tener muchos movimientos.
- Cada movimiento pertenece a **un usuario** y **un activo**.

PROYECTO INTERMODULAR 2

TABLA: LOG_MOVIMIENTOS (versión MVP)

```
DROP TABLE IF EXISTS LOG_MOVIMIENTOS;
CREATE TABLE LOG_MOVIMIENTOS (
   IDMovimiento INT AUTO_INCREMENT PRIMARY KEY,
   FechaHora DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
   TipoAccion ENUM('CREAR', 'EDITAR', 'ELIMINAR', 'CONSULTAR') NOT NULL,
   IDUsuario INT NOT NULL,
   IDActivo INT NOT NULL,
   CONSTRAINT fk_mov_usuario
      FOREIGN KEY (IDUsuario)
      REFERENCES USUARIOS(IDUsuario)
      ON DELETE RESTRICT
      ON UPDATE CASCADE,
   CONSTRAINT fk_mov_activo
      FOREIGN KEY (IDActivo)
      REFERENCES ACTIVOS_DIGITALES(id_activo)
      ON DELETE CASCADE
      ON UPDATE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

ARQUITECTURA DEL MÓDULO

El módulo se compone de los siguientes elementos:

- **Entidad LogMovimiento** → Representa cada acción registrada en el sistema. Contiene:
  - ID del movimiento
  - Fecha y hora
  - Tipo de acción
  - ID del usuario
  - ID del activo afectado

```
package com.miriammendieta.zeerty_digital_app.model;

import jakarta.persistence.*;
import java.time.LocalDateTime;

@Entity
@Table(name = "LOG_MOVIMIENTOS")
public class LogMovimiento {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "IDMovimiento")
    private Integer idMovimiento;

    @Column(name = "FechaHora")
    private LocalDateTime fechaHora;

    @Enumerated(EnumType.STRING)
    @Column(name = "TipoAccion")
```

```
    private TipoAccion tipoAccion;

    @Column(name = "IDUsuario")
    private Integer idUsuario;

    @Column(name = "IDActivo")
    private Integer idActivo;

    // Getters y setters
    public Integer getIdMovimiento() { return idMovimiento; }
    public void setIdMovimiento(Integer idMovimiento) { this.idMovimiento = idMovimiento; }

    public LocalDateTime getFechaHora() { return fechaHora; }
    public void setFechaHora(LocalDateTime fechaHora) { this.fechaHora = fechaHora; }

    public TipoAccion getTipoAccion() { return tipoAccion; }
    public void setTipoAccion(TipoAccion tipoAccion) { this.tipoAccion = tipoAccion; }

    public Integer getIdUsuario() { return idUsuario; }
    public void setIdUsuario(Integer idUsuario) { this.idUsuario = idUsuario; }

    public Integer getIdActivo() { return idActivo; }
    public void setIdActivo(Integer idActivo) { this.idActivo = idActivo; }
}
```

- **DTO/ LogMovimientoDTO**
  El DTO (Data Transfer Object) LogMovimientoDTO se utiliza para transportar información desde el backend hacia la vista de forma clara y legible. Su objetivo es evitar mostrar datos internos como IDs y reemplazarlos por información útil para el auditor:
  - Nombre completo del usuario
  - Rol del usuario
  - Nombre del activo
  - Acción
  - Fecha y hora

```
package com.miriammendieta.zeerty_digital_app.DTO;

import java.time.LocalDateTime;

public class LogMovimientoDTO {

    private Integer id;
    private String usuario;
    private String rolUsuario;
    private String accion;
    private String activoNombre;
    private LocalDateTime fechaHora;

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getUsuario() { return usuario; }
    public void setUsuario(String usuario) { this.usuario = usuario; }

    public String getRolUsuario() { return rolUsuario; }
```

```
    public void setRolUsuario(String rolUsuario) { this.rolUsuario = rolUsuario; }

    public String getAccion() { return accion; }
    public void setAccion(String accion) { this.accion = accion; }

    public String getActivoNombre() { return activoNombre; }
    public void setActivoNombre(String activoNombre) { this.activoNombre = activoNombre; }

    public LocalDateTime getFechaHora() { return fechaHora; }
    public void setFechaHora(LocalDateTime fechaHora) { this.fechaHora = fechaHora; }
}
```

- **Repositorio**
  El repositorio LogMovimientoRepository es el componente encargado de gestionar el acceso a los registros de auditoría almacenados en la base de datos. Está implementado utilizando **Spring Data JPA**, lo que permite trabajar con la entidad LogMovimiento sin necesidad de escribir consultas SQL manuales.

```
package com.miriammendieta.zeerty_digital_app.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.miriammendieta.zeerty_digital_app.model.LogMovimiento;

public interface LogMovimientoRepository extends JpaRepository<LogMovimiento, Integer> {
}
```

Este repositorio actúa como la capa de acceso a datos del módulo de Auditoría. Su responsabilidad es interactuar con la tabla donde se almacenan los movimientos registrados, permitiendo:

- Obtener todos los registros de auditoría.
- Buscar registros por su ID.
- Insertar nuevos movimientos.
- Contar el número total de movimientos, si fuera necesario.

Todo esto sin necesidad de implementar consultas SQL, ya que Spring Data JPA genera automáticamente las operaciones básicas.
Al extender JpaRepository<LogMovimiento, Integer>, el repositorio hereda automáticamente un conjunto completo de métodos CRUD:

- findAll() → obtener todos los registros
- findById(id) →buscar un registro por su ID
- save(entity) →insertar o actualizar
- deleteById(id) → eliminar
- count() → contar registros

Esto elimina la necesidad de implementar consultas SQL manuales, simplificando el desarrollo y reduciendo errores.

❖ **Servicios**

- UsuarioService: obtiene información del usuario.
- ActivoDigitalService: obtiene información del activo.

❖ **Controlador**

El controlador AuditoriaController es el componente responsable de gestionar el acceso al módulo de Auditoría y preparar los datos que se mostrarán en la vista. Su función principal es coordinar el flujo entre la capa de servicios, el repositorio y la interfaz de usuario:

- Validación de sesión: Comprueba que el usuario esté autenticado. Si no lo está, redirige al formulario de inicio de sesión.
- Control de permisos: Solo los roles ADMINISTRACIÓN y AUDITORÍA pueden acceder al registro de movimientos. Si el usuario no tiene permisos, se le redirige a la página principal con un mensaje de error.
- Obtención de registros de auditoría: Utiliza el repositorio LogMovimientoRepository para obtener todos los movimientos registrados en la base de datos.
- Conversión de entidades a DTO: Cada registro LogMovimiento se transforma en un LogMovimientoDTO para mostrar información legible (nombre del usuario, rol y nombre del activo).
- Envío de datos a la vista: Añade la lista de movimientos al modelo para que Thymeleaf pueda renderizar la tabla.

El controlador sigue el patrón **MVC**, separando la lógica de negocio de la presentación. Esto mejora la mantenibilidad y claridad del código.

```
@Controller
@RequestMapping("/registro-movimientos")
public class AuditoriaController {

   private final LogMovimientoRepository logRepo;
   private final UsuarioService usuarioService;
   private final ActivoDigitalService activoService;

   public AuditoriaController(LogMovimientoRepository logRepo,
                              UsuarioService usuarioService,
                              ActivoDigitalService activoService) {
      this.logRepo = logRepo;
```

```
        this.usuarioService = usuarioService;
        this.activoService = activoService;
    }

    @GetMapping("")
    public String verRegistro(Model model, HttpSession session) {

        Usuario usuario = (Usuario) session.getAttribute("usuario");

        if (usuario == null) {
            return "redirect:/login";
        }

        if (usuario.getRol() != Rol.ADMINISTRACION && usuario.getRol() != Rol.AUDITORIA) {
            return "redirect:/home?error=permiso";
        }

        // Cargar logs
        List<LogMovimiento> logs = logRepo.findAll();

        // Convertir logs a DTO con nombres
        List<LogMovimientoDTO> movimientos = new ArrayList<>();

        for (LogMovimiento log : logs) {

            LogMovimientoDTO dto = new LogMovimientoDTO();

            dto.setId(log.getIdMovimiento());
            dto.setFechaHora(log.getFechaHora());
            dto.setAccion(log.getTipoAccion().name());
```

```
        // Usuario
        Usuario u = usuarioService.buscarPorId(log.getIdUsuario());
        dto.setUsuario(u.getNombre() + " " + u.getApellidos());
        dto.setRolUsuario(u.getRol().name());

        // Activo
        ActivoDigital a = activoService.buscarPorId(log.getIdActivo());
        dto.setActivoNombre(a.getNombre());

        movimientos.add(dto);
    }

    model.addAttribute("movimientos", movimientos);
    model.addAttribute("usuario", usuario);

    return "registro-movimientos";
  }
}
```

- **Vista (Thymeleaf + HTML + CSS)**
  La vista del módulo de Auditoría es un template Thymeleaf que se encarga de mostrar los movimientos registrados en el sistema de forma clara, ordenada y visualmente accesible para el usuario con rol ADMINISTRACIÓN o AUDITORÍA. Esta vista combina HTML, CSS y Thymeleaf, permitiendo renderizar dinámicamente los datos enviados desde el controlador.
  registro-movimientos.html muestra los movimientos en una tabla centrada y ordenada.





Panel de gestión vista para el auditor

BIBLIOGRAFÍA

- Apuntes del Profesor.
- Que es un activo Digital: https://www.youtube.com/watch?v=BVIP_qPWzUg
- Auditoría, trazabilidad y seguridad: https://www.iso.org/standard/27001
  https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html
  https://www.youtube.com/shorts/OKPHI13PTyQ
  https://owasp.org/
  https://opencre.org/
  https://martinfowler.com/articles/domain-oriented-observability.html

- **Spring Boot (Backend)**
  Documentación oficial de Spring Boot → https://docs.spring.io/spring-boot/
  Guía oficial de Spring Framework → https://spring.io/projects/spring-framework
  Spring Boot — Building Web Applications → https://spring.io/guides/gs/serving-web-content

- **Spring Data JPA (Repositorios y persistencia)**
  Documentación oficial de Spring Data JPA → https://docs.spring.io/spring-data/jpa/docs/current/reference/html/
  Guía de JPA y Hibernate → https://hibernate.org/orm/documentation/
  Introducción a JPA → https://www.baeldung.com/jpa-hibernate-basics

- **Thymeleaf (Vistas HTML dinámicas)**
  Documentación oficial de Thymeleaf → https://www.thymeleaf.org/documentation.html
  Thymeleaf + Spring Boot Tutorial → https://www.baeldung.com/spring-thymeleaf-template
  Thymeleaf Layouts → https://www.thymeleaf.org/doc/articles/layouts.html

- **Arquitectura MVC y buenas prácticas**
  Patrón MVC explicado → https://developer.mozilla.org/en-US/docs/Glossary/MVC

- **Bases de datos y diseño ER**
  Modelo Entidad-Relación — Conceptos básicos → https://www.ilerna.es/blog/modelo-entidad-relacion-base-datos
  Normalización de bases de datos → https://www.guru99.com/database-normalization.html

- **Documentación**
  Guía de documentación técnica: https://www.arsys.es/blog/hacer-documentacion-tecnica-software
  https://blog.invgate.com/es/documentacion-tecnica
  https://www.getguru.com/es/reference/software-documentation

  Buenas prácticas de documentación de software: https://www.atlassian.com/software/confluence/documentation
  https://asana.com/es/templates/technical-documentation
  https://tecnologia.euroinnova.com/como-documentar-proyectos-de-software